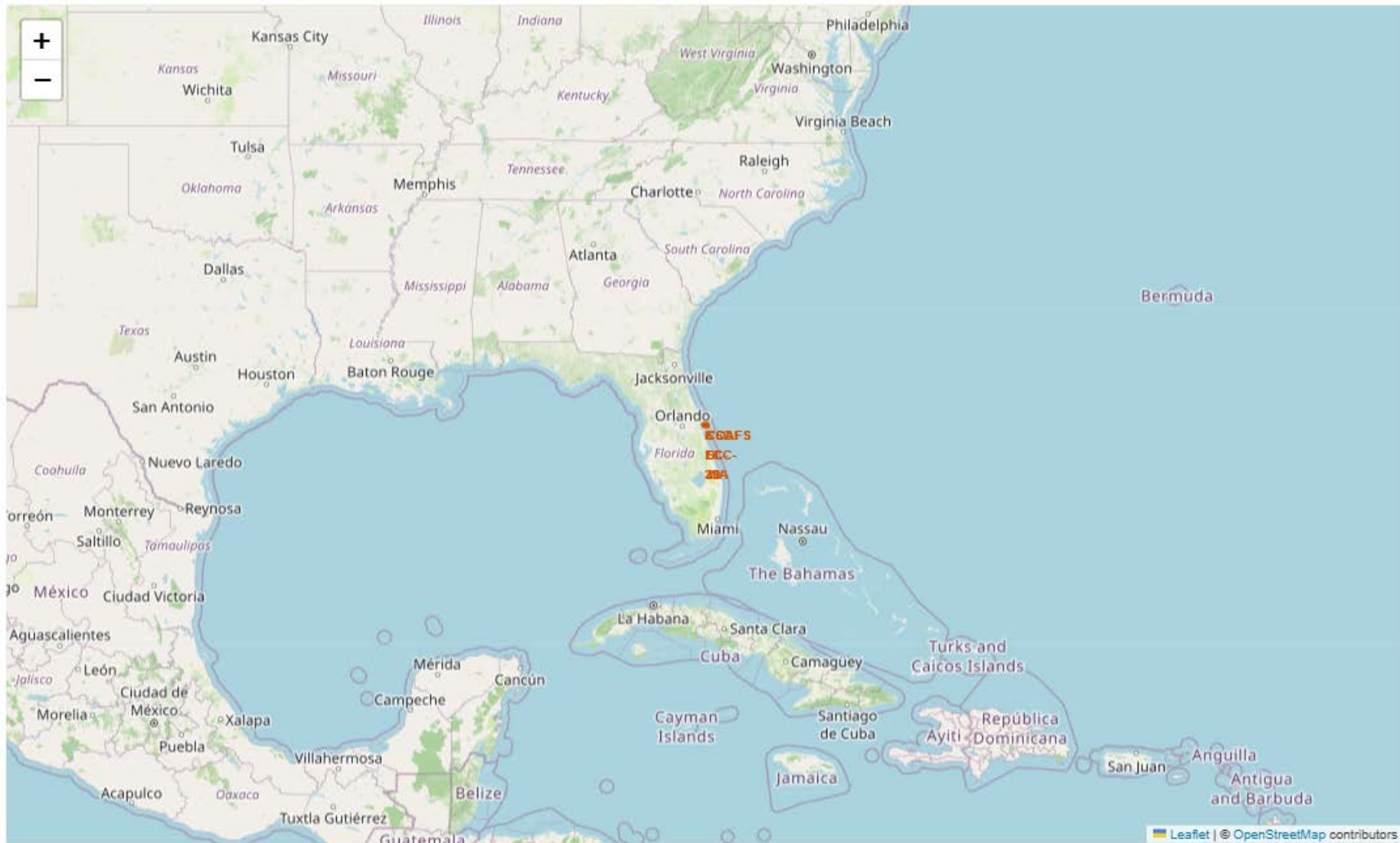


site_map

[5]:



First, let's try to add each site's location on a map using site's latitude and longitude coordinates

The following dataset with the name `spacex_launch_geo.csv` is an augmented dataset with latitude and longitude added for each site.

```

icon_anchor=(0,0),
html='<div style="font-size: 12; color:#d35400;"><b>%s</b></div>' % 'NASA JSC',
)
)

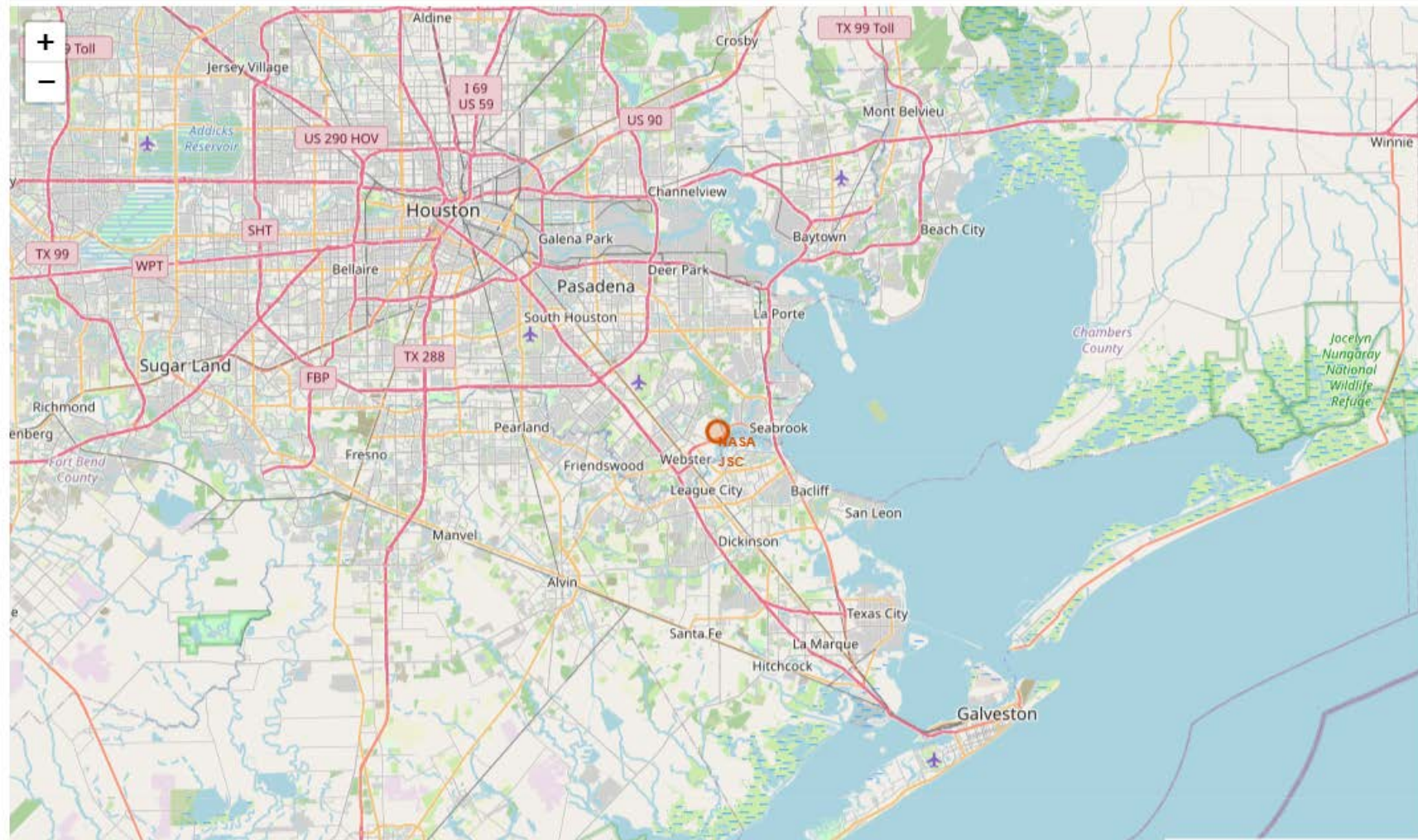
```

```

site_map.add_child(circle)
site_map.add_child(marker)

```

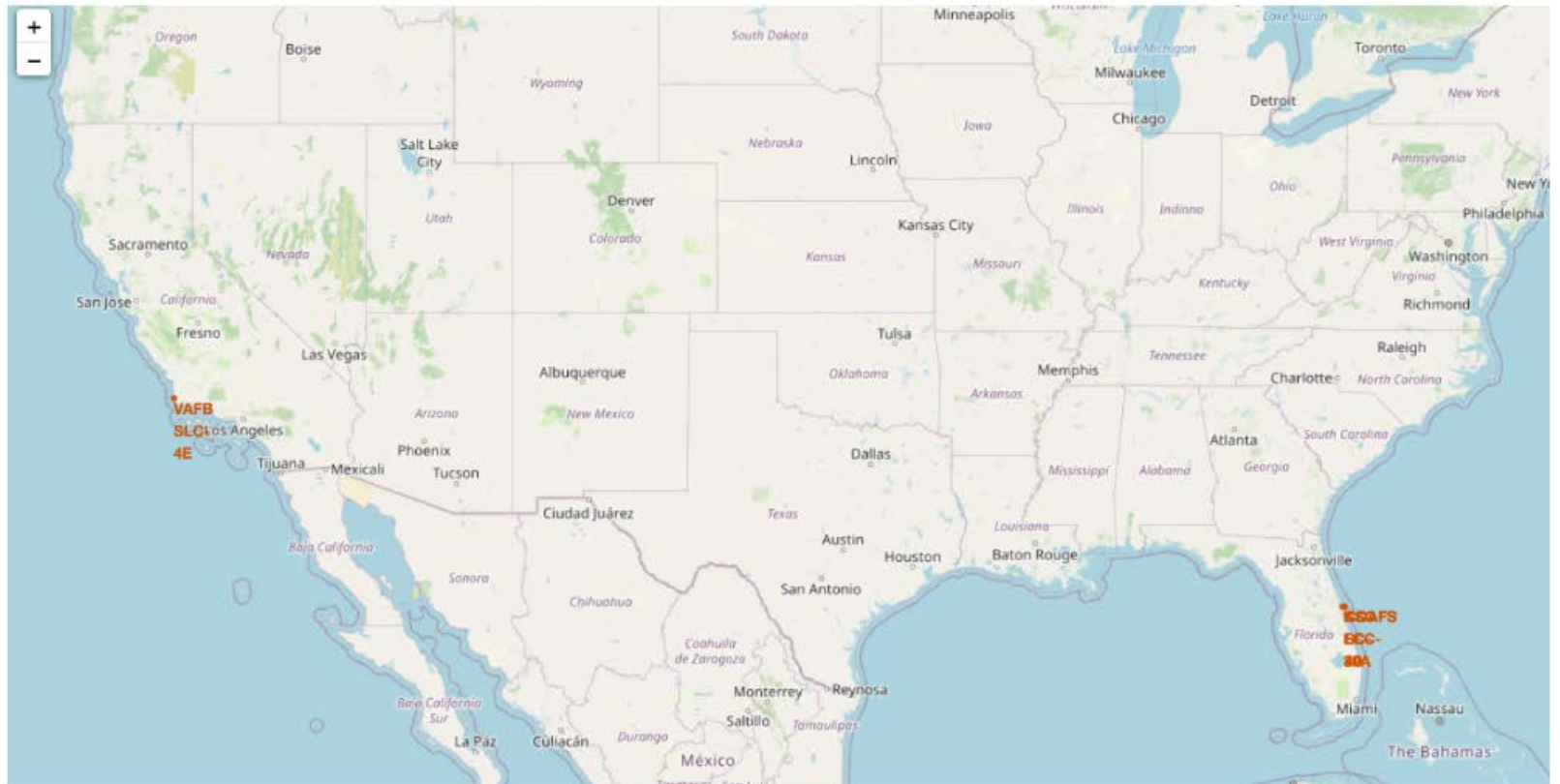
[1]:



```
folium.map.Marker(coordinate, icon=DivIcon(icon_size=(20,20),icon_anchor=(0,0), html='<div style="font-size: 12; color:#d35400;">
<b>%s</b></div>' % 'label', ))
```

```
[ ]: # Initial the map
site_map = folium.Map(location=nasa_coordinate, zoom_start=5)
# For each launch site, add a Circle object based on its coordinate (Lat, Long) values. In addition, add Launch site name as a popup Label
```

The generated map with marked launch sites should look similar to the following:

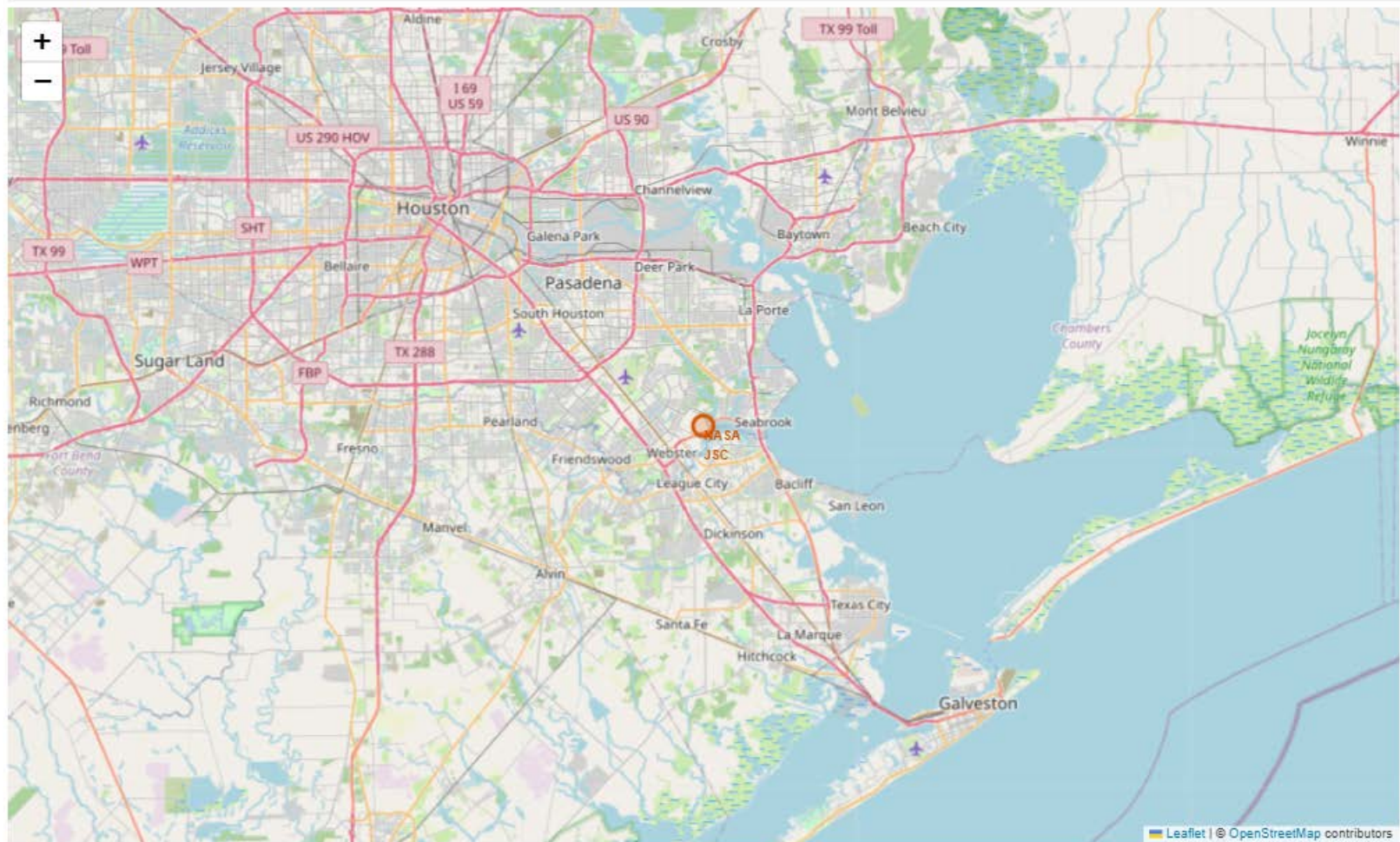


Now, you can explore the map by zoom-in/out the marked areas , and try to answer the following questions:

```
)  
marker_cluster.add_child(marker)
```

```
# Display the map with clustered markers  
site_map
```

12]:



TODO: Create a new column in `spacex_df` dataframe called `marker_color` to store the marker colors based on the `class` value

[]:

```
# Apply a function to check the value of `class` column
```

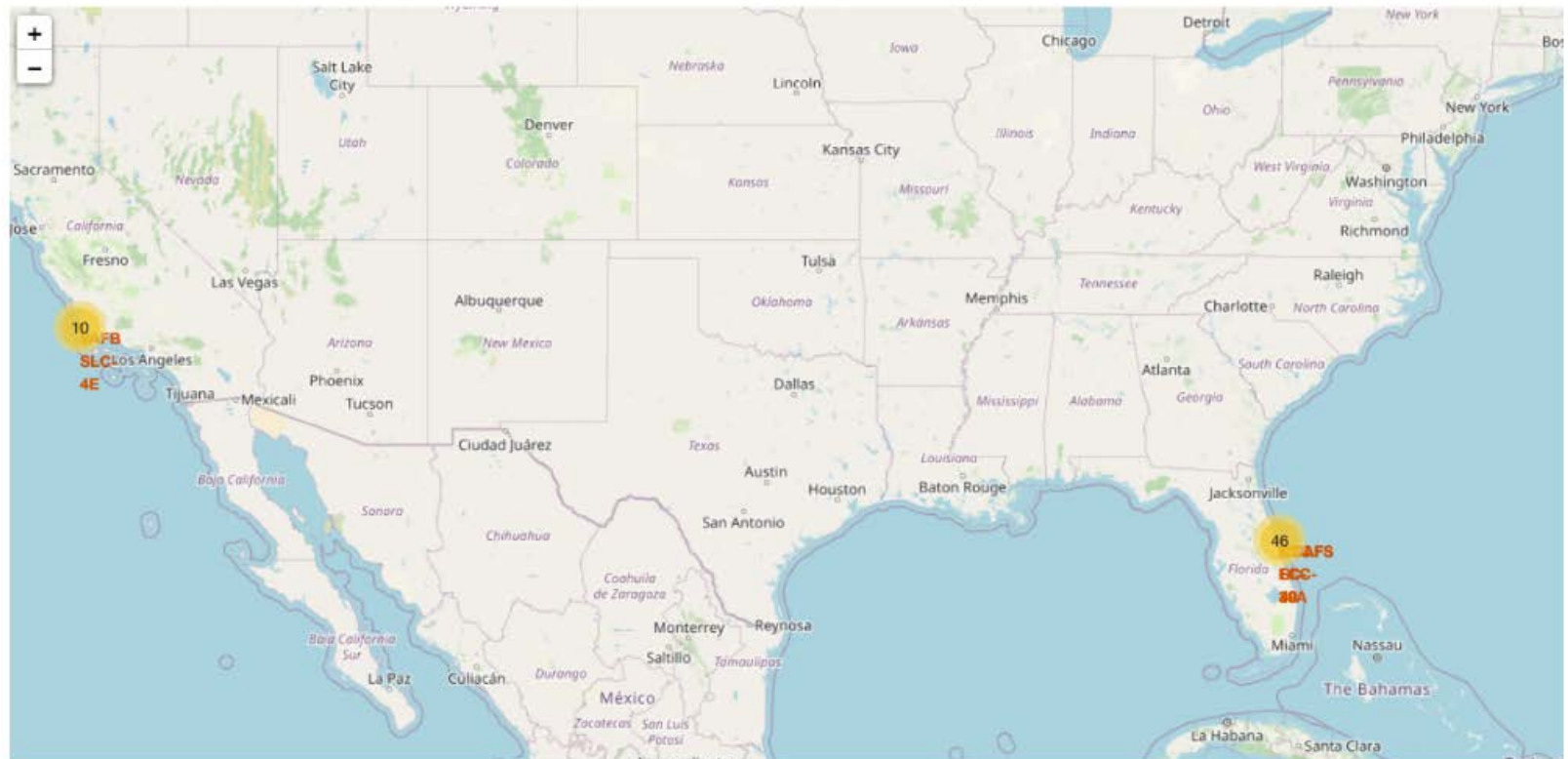
```

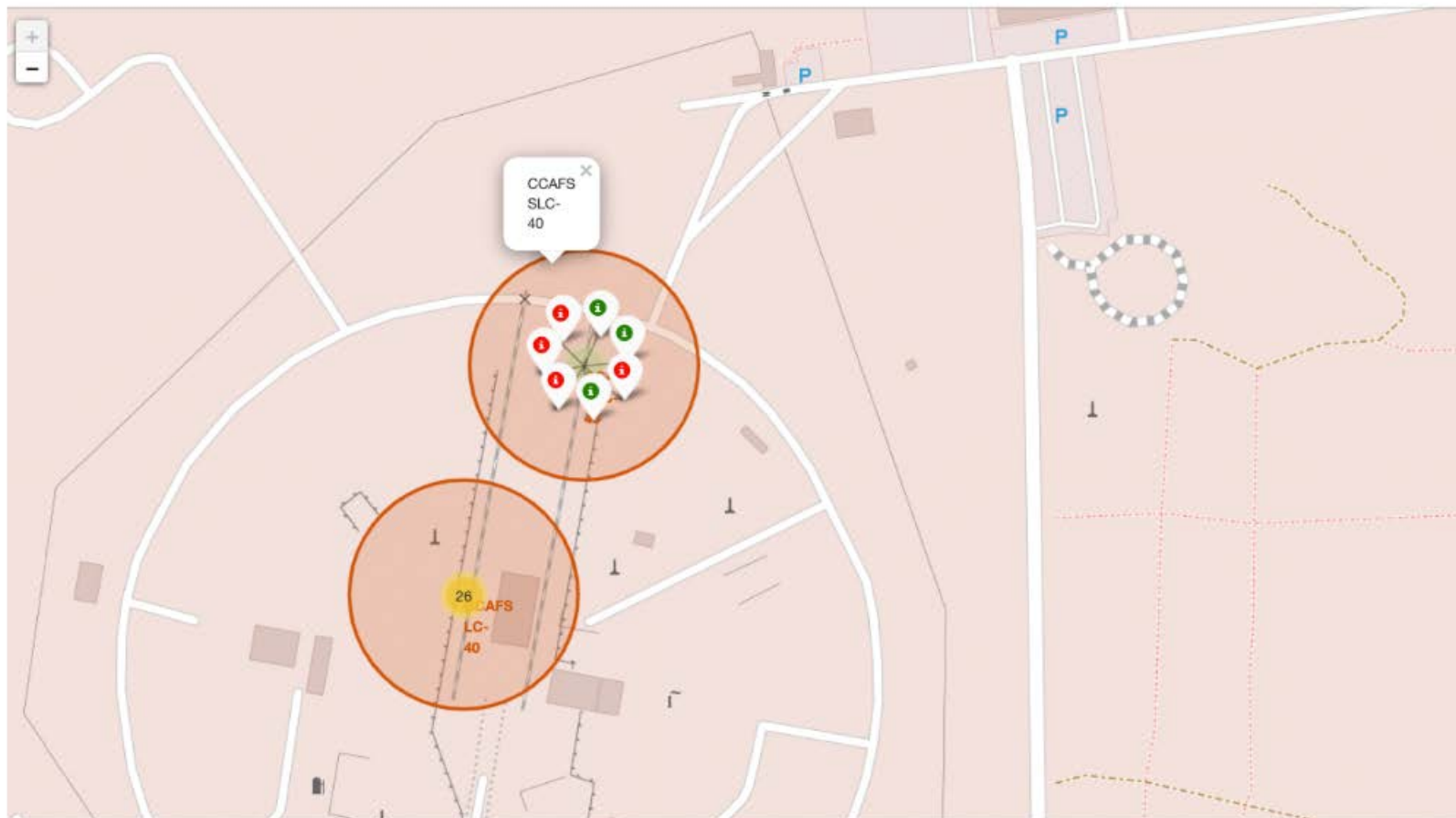
# create a Marker object with its coordinate
# and customize the Marker's icon property to indicate if this launch was succeeded or failed,
# e.g., icon=folium.Icon(color='white', icon_color=row['marker_color'])
for index, record in spacex_df.iterrows():
    # TODO: Create and add a Marker cluster to the site map
    # marker = folium.Marker(...)
    marker_cluster.add_child(marker)

```

site_map

Your updated map may look like the following screenshots:



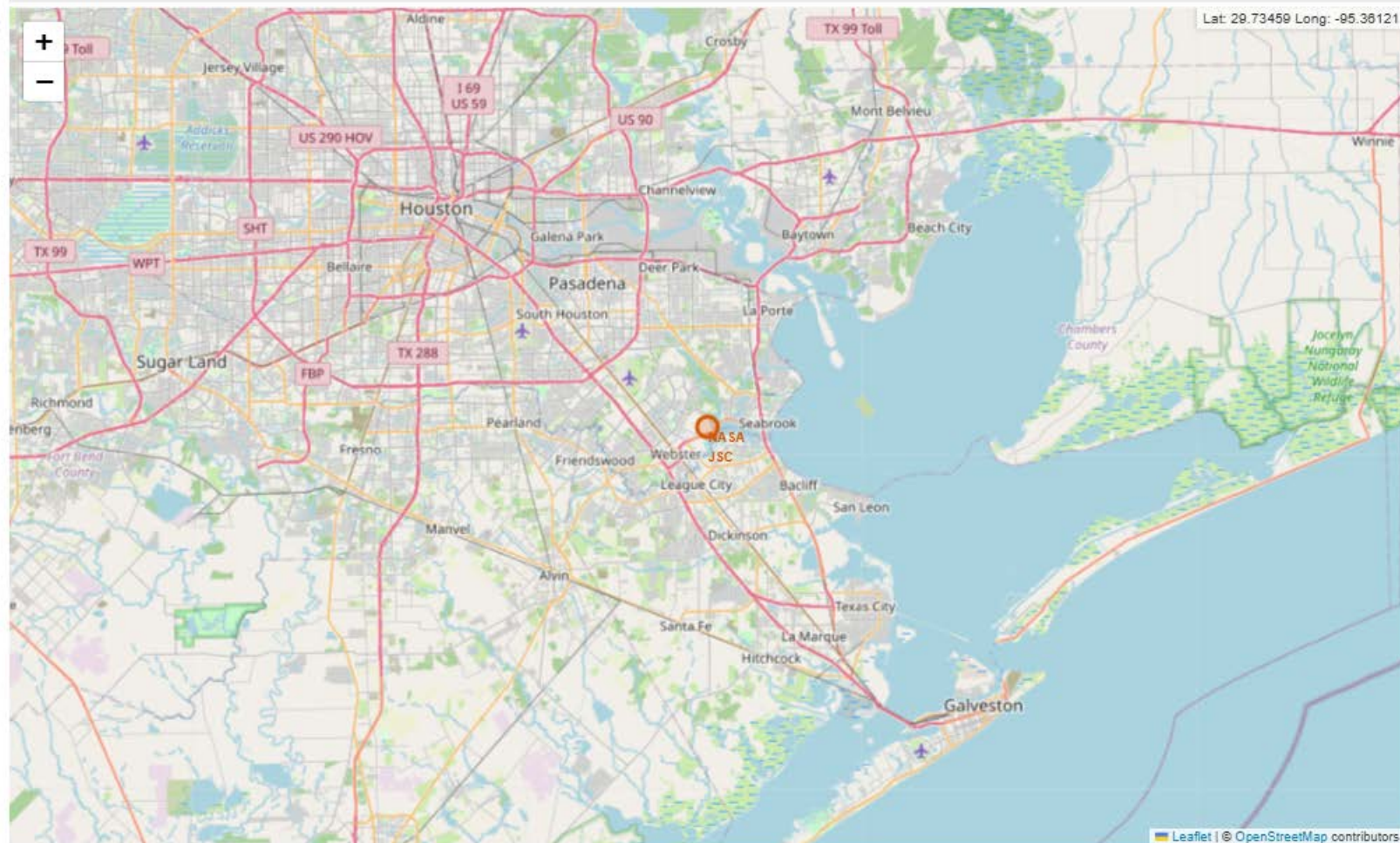


From the color-labeled markers in marker clusters, you should be able to easily identify which launch sites have relatively high success rates.

```
[ ]: # TASK 3: Calculate the distances between a launch site to its proximities
```

```
site_map.add_child(mouse_position)
site_map
```

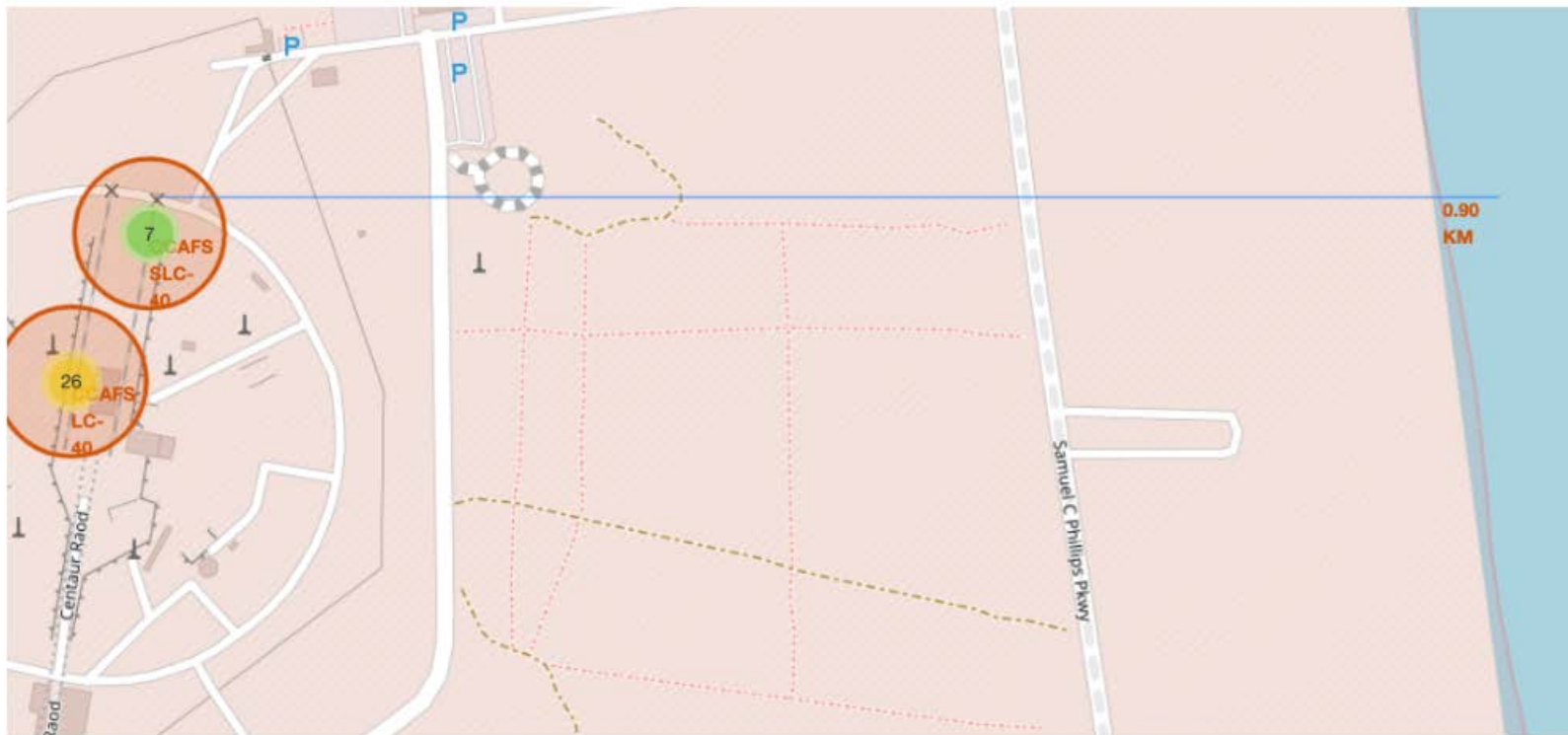
[13]:



Now zoom in to a launch site and explore its proximity to see if you can easily find any railway, highway, coastline, etc. Move your mouse to these points and mark down their coordinates (shown on the top-left) in order to the distance to the launch site.

```
[ ]: # Create a `folium.PolyLine` object using the coastline coordinates and launch site coordinate
# lines=folium.PolyLine(locations=coordinates, weight=1)
site_map.add_child(lines)
```

Your updated map with distance line should look like the following screenshot:

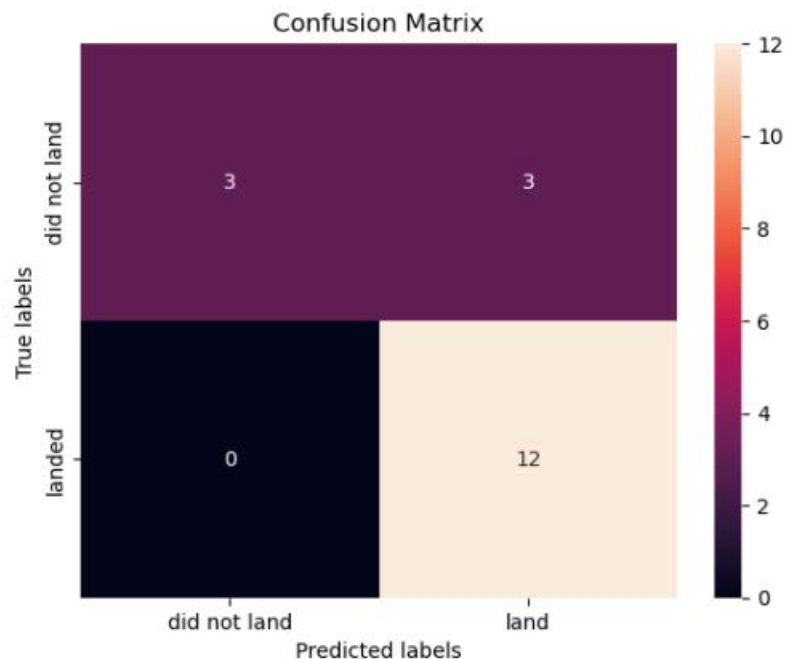


TODO: Similarly, you can draw a line between a launch site to its closest city, railway, highway, etc. You need to use `MousePosition` to find the their coordinates on the map first

```
# 4. Define parameters and fit model
parameters = {'C': [0.01, 0.1, 1],
              'penalty': ['l2'],
              'solver': ['lbfgs']}

lr = LogisticRegression()
logreg_cv = GridSearchCV(lr, parameters, cv=10)
logreg_cv.fit(X_train, Y_train)

# 5. Make predictions and display matrix
yhat = logreg_cv.predict(X_test)
plot_confusion_matrix(Y_test, yhat)
```



Load the dataframe

Load the data

```
data = pd.read_csv(URL1)
```

```
[9]: data.head()
```

```
[9]:
```

	FlightNumber	Date	BoosterVersion	PayloadMass	Orbit	LaunchSite	Outcome	Flights	GridFins	Reused	Legs	LandingPad	Block	ReusedCount	Serial	Long
0	1	2010-06-04	Falcon 9	6104.959412	LEO	CCAFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B0003	-80.5'
1	2	2012-05-22	Falcon 9	525.000000	LEO	CCAFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B0005	-80.5'
2	3	2013-03-01	Falcon 9	677.000000	ISS	CCAFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B0007	-80.5'
3	4	2013-09-29	Falcon 9	500.000000	PO	VAFB SLC 4E	False Ocean	1	False	False	False	NaN	1.0	0	B1003	-120.6
4	5	2013-12-03	Falcon 9	3170.000000	GTO	CCAFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B1004	-80.5'

```
[23]: URL2 = 'https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DS0321EN-SkillsNetwork/datasets/dataset_part_3.csv'
X = pd.read_csv(URL2)
```

```
[24]: X.head(100)
```

```
[24]:
```

	FlightNumber	PayloadMass	Flights	Block	ReusedCount	Orbit_ES- L1	Orbit_GEO	Orbit_GTO	Orbit_HEO	Orbit_ISS	...	Serial_B1058	Serial_B1059	Serial_B1060	S
0	1.0	6104.959412	1.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	
1	2.0	525.000000	1.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	
2	3.0	677.000000	1.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0	...	0.0	0.0	0.0	
3	4.0	500.000000	1.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	
4	5.0	3170.000000	1.0	1.0	0.0	0.0	0.0	1.0	0.0	0.0	...	0.0	0.0	0.0	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	
85	86.0	15400.000000	2.0	5.0	2.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	1.0	
86	87.0	15400.000000	3.0	5.0	2.0	0.0	0.0	0.0	0.0	0.0	...	1.0	0.0	0.0	
87	88.0	15400.000000	6.0	5.0	5.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	
88	89.0	15400.000000	3.0	5.0	2.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	1.0	

TASK 4

Create a logistic regression object then create a GridSearchCV object `logreg_cv` with `cv = 10`. Fit the object to find the best parameters from the dictionary `parameters`.

```
[28]: parameters = {'C':[0.01,0.1,1],  
                  'penalty':['l2'],  
                  'solver':['lbfgs']}  
lr = LogisticRegression()  
logreg_cv = GridSearchCV(lr, parameters, cv=10)  
logreg_cv.fit(X_train, Y_train)
```

```
[28]: > GridSearchCV  
      > best_estimator_:  
          LogisticRegression  
        > LogisticRegression
```

```
[29]: parameters = {'C':[0.01,0.1,1], 'penalty':['l2'], 'solver':['lbfgs']}# l1 lasso l2 ridge  
lr=LogisticRegression()
```

We output the `GridSearchCV` object for logistic regression. We display the best parameters using the data attribute `best_params_` and the accuracy on the validation data using the data attribute `best_score_`.

```
[30]: print("tuned hyperparameters :(best parameters) ",logreg_cv.best_params_)  
      print("accuracy :",logreg_cv.best_score_)  
  
tuned hyperparameters :(best parameters) {'C': 0.01, 'penalty': 'l2', 'solver': 'lbfgs'}  
accuracy : 0.8464285714285713
```

TASK 5

Calculate the accuracy on the test data using the method `score`:

```
[31]: logreg_score = logreg_cv.score(X_test, Y_test)  
      print("Logistic Regression Test Accuracy:", logreg_score)
```

TASK 5

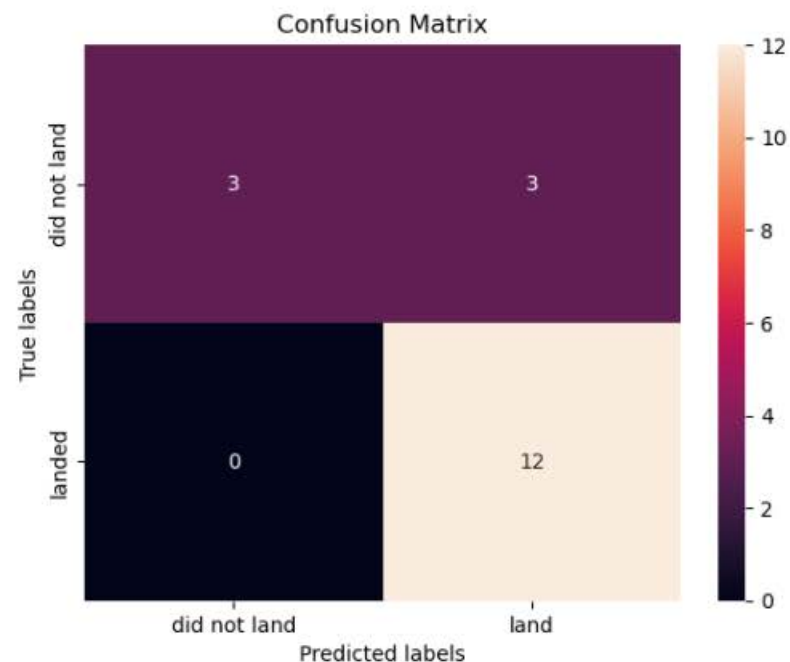
Calculate the accuracy on the test data using the method `score` :

```
[31]: logreg_score = logreg_cv.score(X_test, Y_test)  
print("Logistic Regression Test Accuracy:", logreg_score)
```

Logistic Regression Test Accuracy: 0.8333333333333334

Lets look at the confusion matrix:

```
[32]: yhat=logreg_cv.predict(X_test)  
plot_confusion_matrix(Y_test,yhat)
```



Examining the confusion matrix, we see that logistic regression can distinguish between the different classes. We see that the problem is false positives.

TASK 6

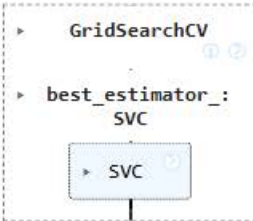
Create a support vector machine object then create a `GridSearchCV` object `svm_cv` with `cv = 10`. Fit the object to find the best parameters from the dictionary `parameters`.

```
[33]: parameters = {'kernel':('linear', 'rbf', 'poly', 'rbf', 'sigmoid'),
                  'C': np.logspace(-3, 3, 5),
                  'gamma':np.logspace(-3, 3, 5)}
svm = SVC()
```

```
[34]: parameters = {'kernel':('linear', 'rbf', 'poly', 'sigmoid'),
                  'C': np.logspace(-3, 3, 5),
                  'gamma': np.logspace(-3, 3, 5)}

svm = SVC()
svm_cv = GridSearchCV(svm, parameters, cv=10)
svm_cv.fit(X_train, Y_train)
```

```
[34]:
```



```
[35]: print("tuned hpyerparameters :(best parameters) ",svm_cv.best_params_)
      print("accuracy :",svm_cv.best_score_)
```

```
tuned hpyerparameters :(best parameters) {'C': np.float64(1.0), 'gamma': np.float64(0.03162277660168379), 'kernel': 'sigmoid'}
accuracy : 0.8482142857142856
```

TASK 7

Calculate the accuracy on the test data using the method `score`:

```
[36]: svm_score = svm_cv.score(X_test, Y_test)
      print("SVM Test Accuracy:", svm_score)
```

```
SVM Test Accuracy: 0.8333333333333334
```

We can plot the confusion matrix

```
[38]: print("tuned hpyerparameters :(best parameters) ",tree_cv.best_params_)
print("accuracy :",tree_cv.best_score_)

tuned hpyerparameters :(best parameters) {'criterion': 'entropy', 'max_depth': 2, 'max_features': 'sqrt', 'min_samples_leaf': 2, 'min_samples_split':
10, 'splitter': 'random'}
accuracy : 0.8732142857142857
```

TASK 9

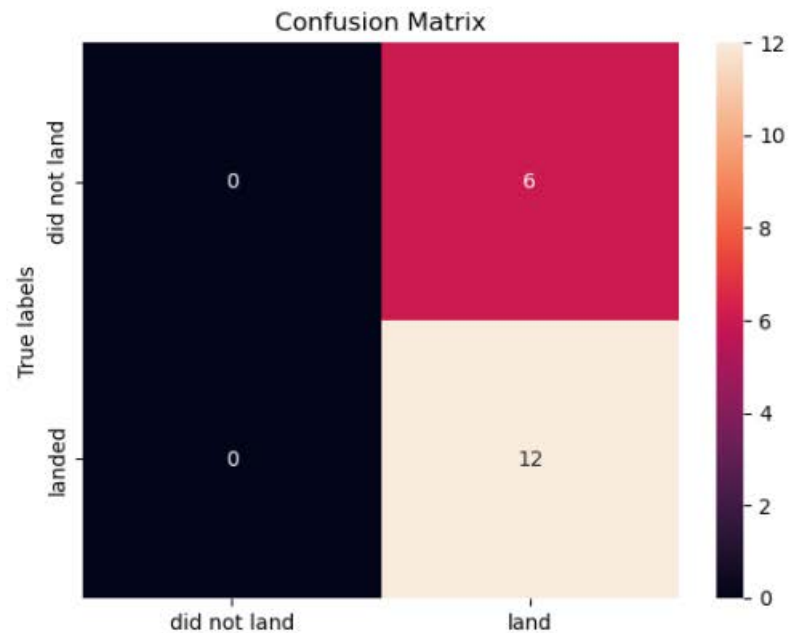
Calculate the accuracy of tree_cv on the test data using the method `score` :

```
[39]: tree_score = tree_cv.score(X_test, Y_test)
print("Decision Tree Test Accuracy:", tree_score)

Decision Tree Test Accuracy: 0.6666666666666666
```

We can plot the confusion matrix

```
[40]: yhat = tree_cv.predict(X_test)
plot_confusion_matrix(Y_test,yhat)
```



TASK 10

Create a k nearest neighbors object then create a `GridSearchCV` object `knn_cv` with `cv = 10`. Fit the object to find the best parameters from the dictionary `parameters`.

```
[ ]: parameters = {'n_neighbors': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],  
                  'algorithm': ['auto', 'ball_tree', 'kd_tree', 'brute'],  
                  'p': [1,2]}
```

```
KNN = KNeighborsClassifier()
```

```
[41]: parameters = {'n_neighbors': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],  
                  'algorithm': ['auto', 'ball_tree', 'kd_tree', 'brute'],  
                  'p': [1,2]}
```

```
KNN = KNeighborsClassifier()
```

```
knn_cv = GridSearchCV(KNN, parameters, cv=10)
```

```
knn_cv.fit(X_train, Y_train)
```

```
[41]:
```

```
  > GridSearchCV  
  
  > best_estimator_:  
    KNeighborsClassifier  
      > KNeighborsClassifier
```

```
[42]: print("tuned hyperparameters :(best parameters) ",knn_cv.best_params_)  
      print("accuracy :",knn_cv.best_score_)
```

```
tuned hyperparameters :(best parameters) {'algorithm': 'auto', 'n_neighbors': 10, 'p': 1}  
accuracy : 0.8482142857142858
```

TASK 11

Calculate the accuracy of `knn_cv` on the test data using the method `score`: